

Zoekalgoritmes

Zoeken, een introductie

- Zoeken is een fundamenteel deel van bijna elke probleemoplossende strategie
- Eigenlijk is 'zoekalgoritme' niet de best naam. 'Vindalgoritme' zou beter zijn.
- Als de agent alle informatie over het probleem kent:
 - Er kan een doel worden geformuleerd
 - Het probleem kan worden geformuleerd
 - Er kan worden **gezocht**
 - Als de beste strategie is worden gevonden, kan deze worden uitgevoerd

Zoeken, een introductie

- Een zoekprobleem:
 - Heeft een eindige hoeveelheid **states**, in een zogenaamde **state space**
 - Heeft een **initiële state**
 - Heeft minstens 1 **goal state**
- Een zoekprobleem heeft ook:
 - **Acties**
 - Een **transitiemodel**, dus voor elke actie op een staat is het resultaat gekend
 - Een **actie-kost functie**
 - Hiermee kan je van de ene naar de andere state gaan.

Volledig observeerbaar, deterministisch

- In een volledig observeerbaar deterministisch probleem, is de oplossing altijd:
 - Voer bepaalde handelingen in de juiste volgorde uit
 - Deze reeks handelingen noemen we een **pad**
 - Een **oplossing** is een **pad** dat begint in de **begin state** en eindigt in de **goal state**
 - Een **optimale oplossing** is een oplossing met de laagste totale actiekost
- We beperken ons nu even tot dit soort problemen.

Voorbeeld: van Antwerpen naar Parijs

- We willen (met de auto) van Antwerpen naar Parijs
- We gaan het probleem wat versimpelen om in plaats van exacte wegen, ons te beperken tot grote steden waar we langs rijden
- States ?
 - Initial ?
 - Final
- Acties ?
 - Transitiemodel ?
- Wat is een goede kost-functie voor dit probleem ?

Voorbeeld: van Antwerpen naar Parijs

- **States:** {Antwerpen, Eindhoven, Gent, Breda, Rotterdam, Brussel, ... Versailles, **Parijs**}
- States ?
 - Initial ?
 - Final
- Acties ?
 - Transitiemodel ?
- Wat is een goede kost-functie voor dit probleem ?



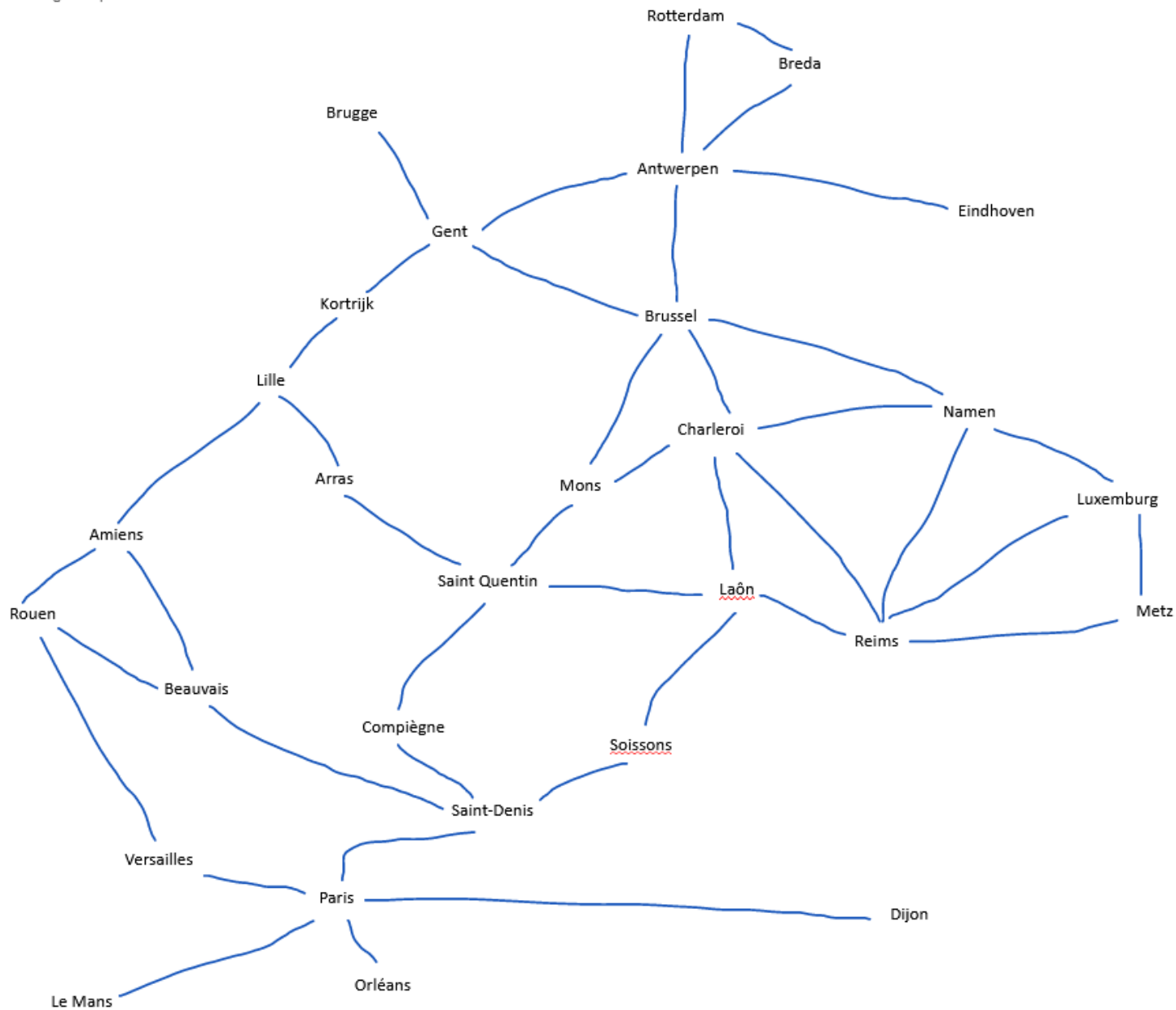
A map showing the geographical distribution of cities in Belgium and France. The cities are labeled as follows:

- Belgium: Brugge, Antwerpen, Gent, Kortrijk, Brussel, Charleroi, Mons, Namen, Eindhoven, Breda, Rotterdam.
- France: Rouen, Amiens, Arras, Lille, Saint-Quentin, Compiègne, Saint-Denis, Versailles, Paris, Orléans, Le Mans, Soissons, Reims, Metz, Luxembourg.

The cities are distributed across the map, with Belgium cities generally located in the northern and central parts, and French cities extending further south and west. The cities of Laon and Soissons are highlighted with red dashed lines.

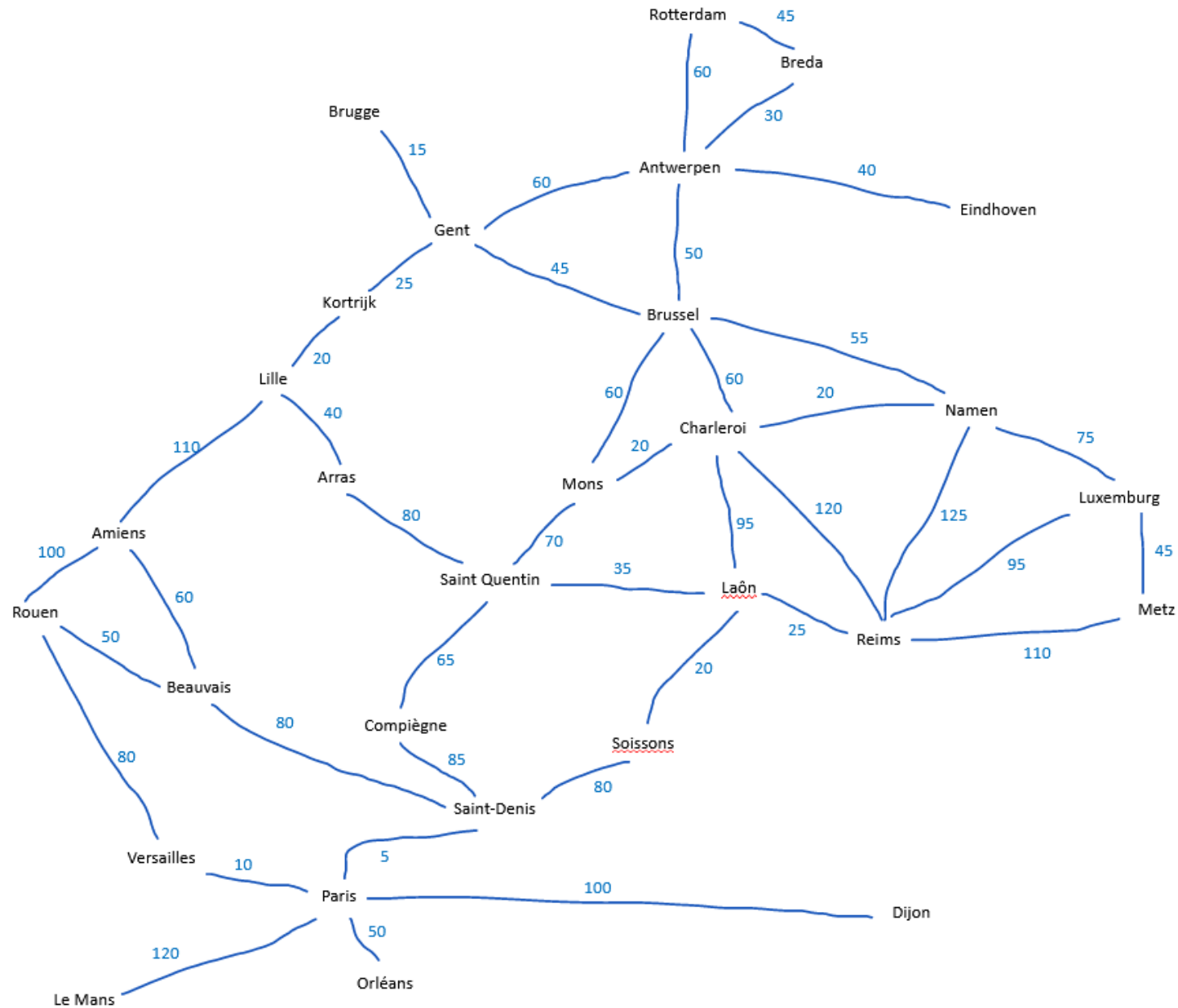
Voorbeeld: van Antwerpen naar Parijs

- **States:** {Antwerpen, Eindhoven, Gent, Breda, Herentals, Brussel, ... Versailles, **Parijs**}
- Een actie is het verplaatsen van bijvoorbeeld Antwerpen naar Gent
- Het transitiemodel bevat dus een **graaf** van alle steden zoals die met elkaar zijn verbonden
- States ?
 - Initial ?
 - Final
- Acties ?
 - Transitiemodel ?
- Wat is een goede kost-functie voor dit probleem ?



Voorbeeld: van Antwerpen naar Parijs

- Er zijn verschillende goede kostfuncties:
 - Reistijd
 - Afstand
 - Verwacht energieverbruik
 - CO2 uitstoot
 - Combinatie van bovenstaande
- States ?
 - Initial ?
 - Final
- Acties ?
 - Transitie-model ?
- Wat is een goede kostfunctie voor dit probleem ?



Soorten zoekstrategieën

Ongeïnformeerd:

hier kent het algoritme enkel de graaf van het probleem.

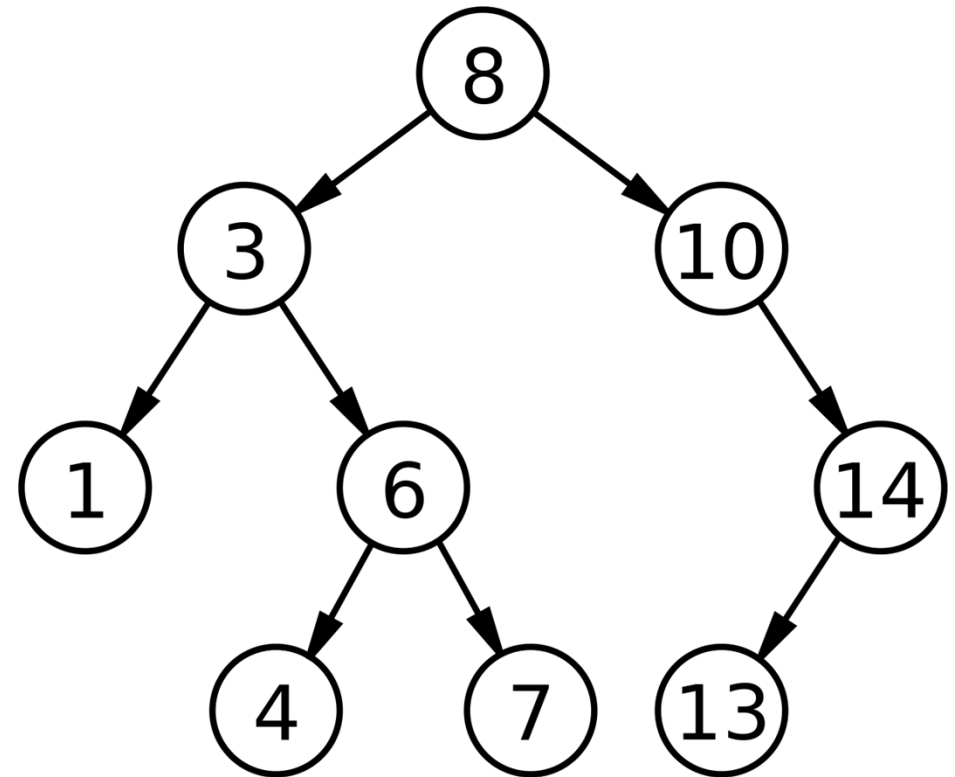
De algoritmes zijn heel algemeen werkend.

Geïnformeerd

Hier kent het algoritme ook 'heuristieken' of truukjes om bepaalde dingen slimmer te doen. Deze algoritmes zijn sneller maar specifiek

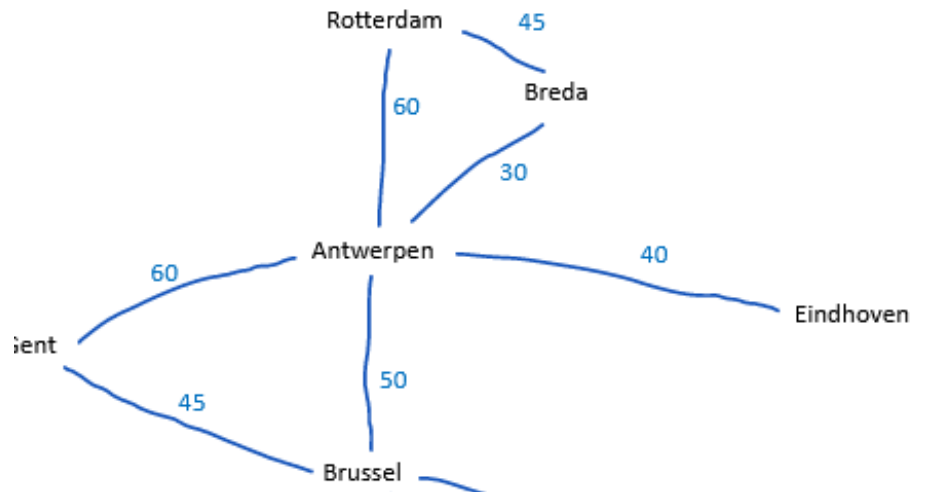
Ongeïnformeerde zoekalgoritmes

- Deze zoekalgoritmes bouwen een **search tree** op basis van de state graph.
- Een node in de search tree is een state in de state graph, en de subnodes van een node moeten in de state graph rechtstreeks zijn verbonden
- De vraag is met welke node we beginnen. Dit resulteert in verschillende algoritmes.



Ongeïnformeerde zoekalgoritmes

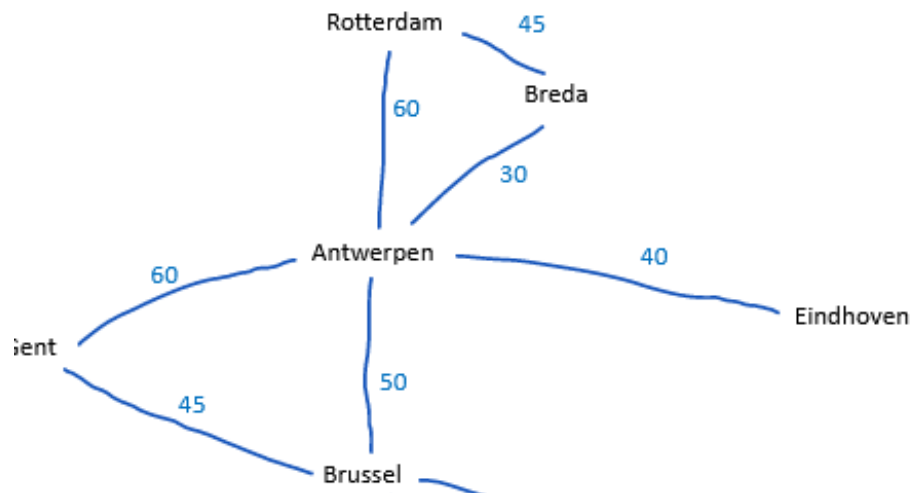
- Welke node klappen we eerst open ?
 - Breadth-First
 - Depth-First



Ongeïnformeerde zoekalgoritmes: breadth-first

Boomstructuur **search tree**:

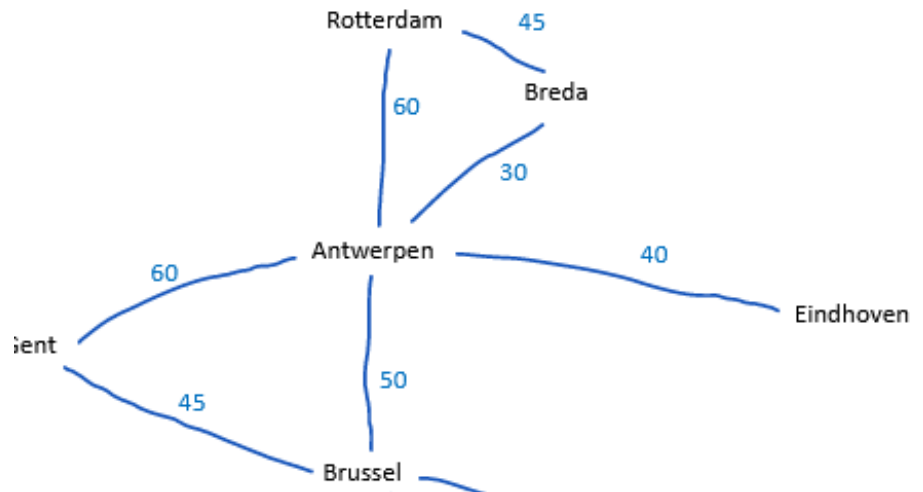
- Antwerpen
 - Rotterdam (60)
 - Breda (30)
 - Eindhoven (40)
 - Brussel (50)
 - Gent (60)
- Deze 5 steden zijn nu de **frontier**: ze zijn nog niet verder uitgeklapt
- Deze 5 steden zijn ook **bereikt**, we kennen de kost ervan



Ongeïnformeerde zoekalgoritmes: breadth-first

Boomstructuur **search tree**:

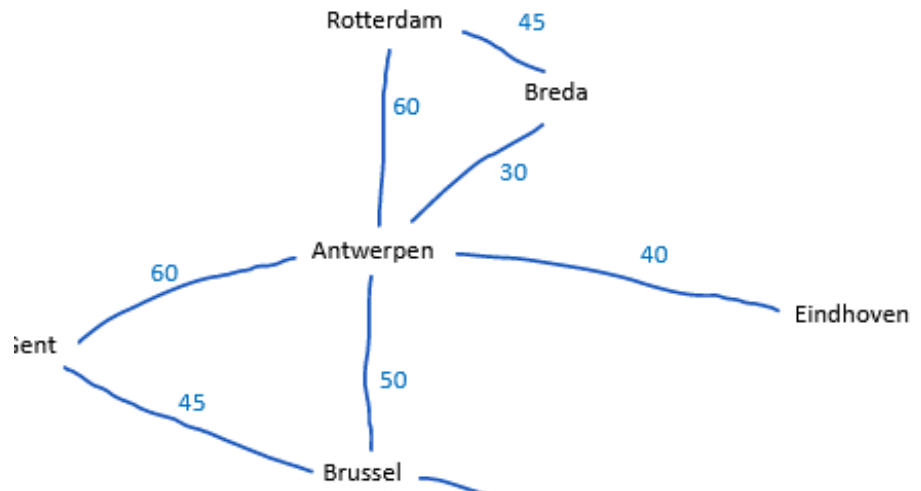
- Antwerpen
 - Rotterdam (60)
 - ~~Antwerpen~~ (LOOP)
 - Breda (45)
 - Breda (30)
 - Eindhoven (40)
 - Brussel (50)
 - Gent (60)
- **Frontier**: Breda (langs Rotterdam), Breda langs Antwerpen, Eindhoven, Brussel, Gent
- **Bereikt**: Rdam, Breda, Eindh, Bru, Gent



Ongeïnformeerde zoekalgoritmes: breadth-first

Boomstructuur **search tree**:

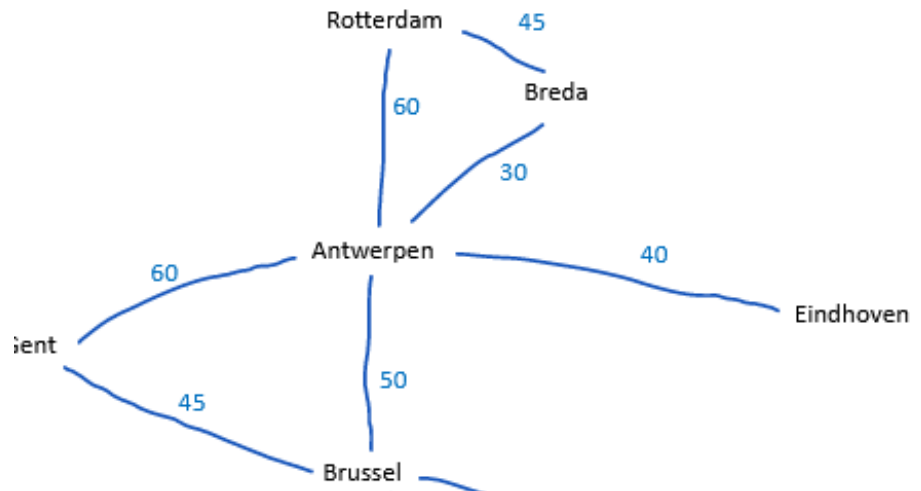
- Antwerpen
 - Rotterdam (60)
 - ~~Antwerpen~~ (LOOP)
 - Breda (45)
 - Breda (30)
 - Rotterdam (45)
 - ~~Antwerpen~~ (LOOP)
 - Eindhoven (40)
 - Brussel (50)
 - Gent (60)
- **Frontier**: Breda (langs Rotterdam), Breda langs Antwerpen, Eindhoven, Brussel, Gent



Ongeïnformeerde zoekalgoritmes: breadth-first

Boomstructuur **search tree**:

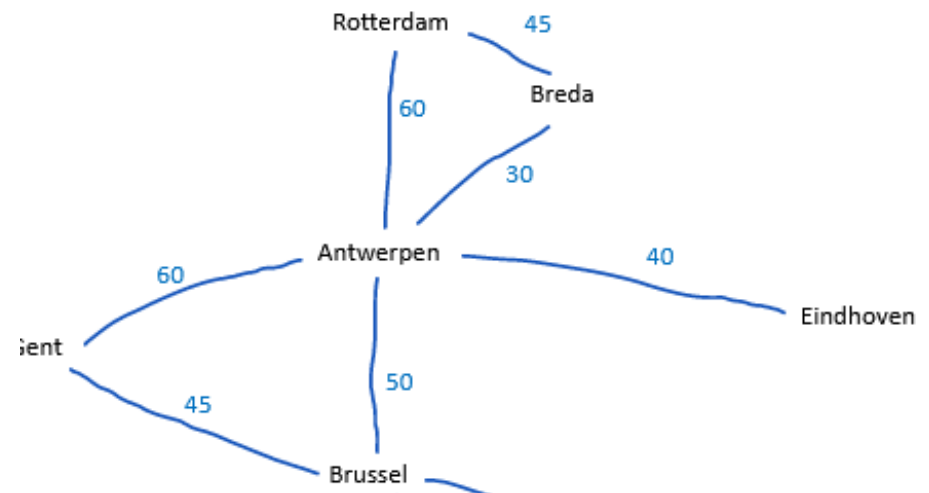
- Antwerpen
 - Rotterdam (60)
 - ~~Antwerpen~~ (LOOP)
 - Breda (45)
 - Breda (30)
 - Rotterdam (45)
 - ~~Antwerpen~~ (LOOP)
 - Eindhoven (40)
 - ~~Antwerpen~~ (LOOP)
 - Brussel (50)
 - Gent (60)
- **Frontier**: Breda (langs Rotterdam), Breda langs Antwerpen, Eindhoven, Brussel, Gent



Ongeïnformeerde zoekalgoritmes: breadth-first

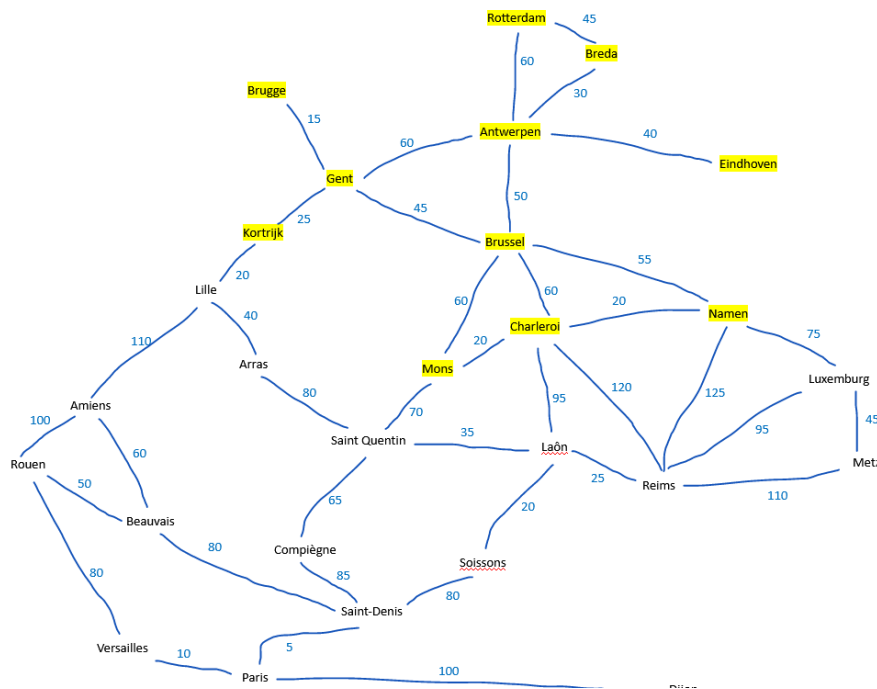
Boomstructuur **search tree**:

- Antwerpen
 - Rotterdam (60)
 - ~~Antwerpen~~ (LOOP)
 - Breda (45)
 - Breda (30)
 - Rotterdam (45)
 - ~~Antwerpen~~ (LOOP)
 - Eindhoven (40)
 - ~~Antwerpen~~ (LOOP)
 - Brussel (50)
 - Gent (45)
 - Namen (55)
 - Charleroi (60)
 - Mons (60)
 - Gent (60)
- **Frontier**: Breda (langs Rotterdam), Breda langs Antwerpen, Eindhoven, Brussel, Gent



Ongeïnformeerde zoekalgoritmes: breadth-first

Dit zijn de bereikte steden na 2 levels vanuit Antwerpen te hebben opengeklapt



Het is duidelijk dat we uiteindelijk in Parijs geraken, maar dat er andere, mogelijke slimmere, strategieën zijn.

Ongeinformeerde zoekalgoritmes: depth-first

- In plaats van in volgorde alle naburige steden verder te onderzoeken, zou je ook eerst in de diepte kunnen zoeken:
 - Je bekijkt eerst de meest veelbelovende node, en gaat die eerst helemaal onderzoeken – je neemt dus de diepste node uit de frontiers, en klapt die open
- Dit leidt sneller tot een uitkomst, maar je riskeert de allerbeste oplossing te missen (waarom ?)
- Het heeft veel minder geheugen nodig dan depth first
 - Het is de **standaardkeuze** in heel veel gevallen

Ongeïnformeerde zoekalgoritmes

- **Breadth first**

- Relevant wanneer de onderlinge child nodes in een graph dezelfde kost hebben
- **Systematisch**: elke steen wordt omgedraaid, niks wordt vergeten
- Het vindt de oplossing in het minste aantal stappen op voorwaarde dat de kost overal hetzelfde is
- **Complexiteit**:
 - b : branching number: aantal opties per node
 - d : diepte van de graph, dus in ons geval aantal steden tussen Antwerpen en Parijs
 - $O(b^d)$
 - Groeit dus heel hard!
- In de praktijk enkel bruikbaar voor kleine depths !!

- **Depth First**

- De standaardkeuze in alle andere gevallen
- Tenzij je meer info hebt over het probleem, en je nog slimmer de eerste node kan kiezen.

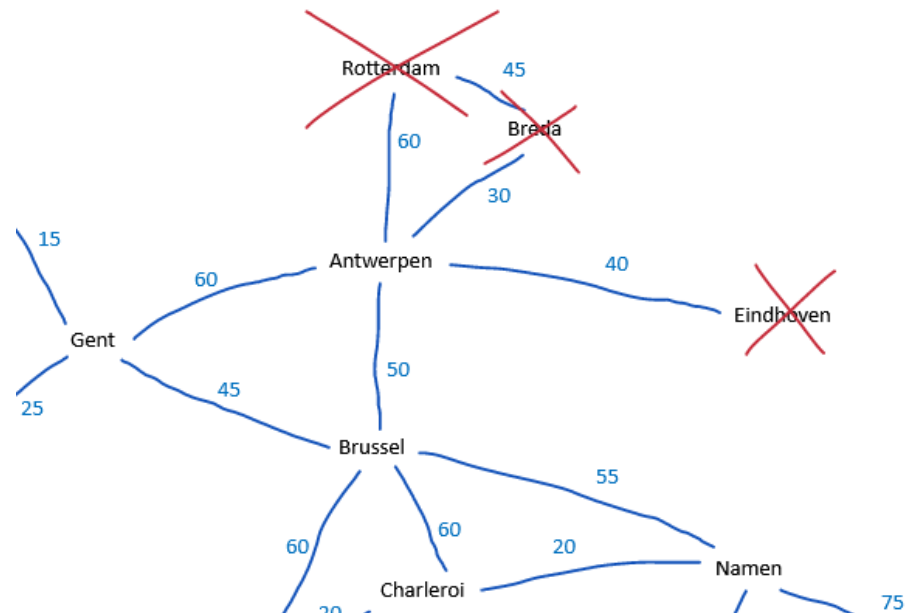
Implementatie

demo

Geïnformeerde zoekalgoritmes: best-first

Wat is ons probleem een goede keuze voor 'best' ?

- In vogelvlucht dichterbij is een kandidaat functie, als de coördinaten van de steden natuurlijk gekend zijn



Het eerste spel: een schuifpuzzel

- We gaan een schuifpuzzel oplossen door een zoekalgoritme.
- Denk al is na over
 - Het probleem, de schuifpuzzel, hoe dit implementeren ?



Slide 75

HBO

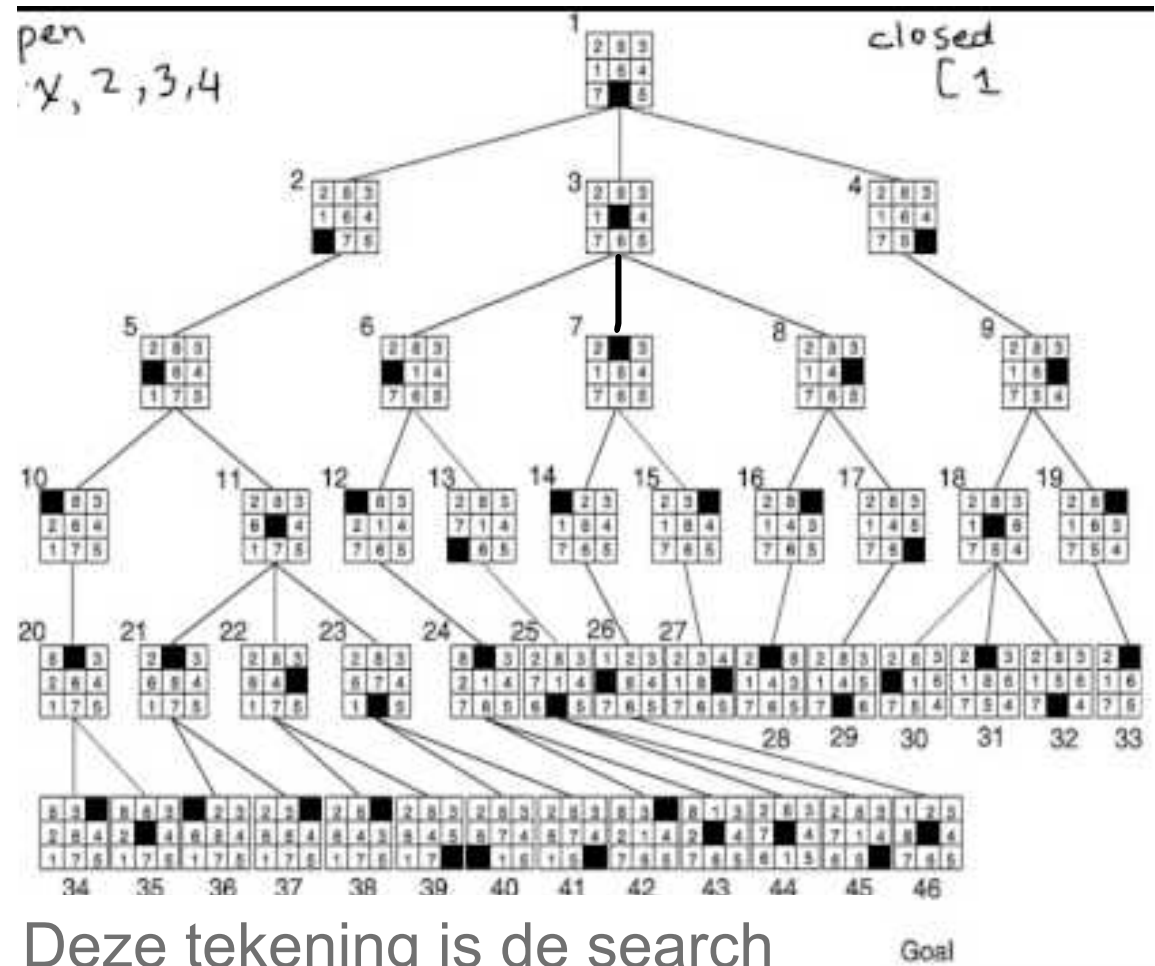
Moet denk ik iets later, bij de lokale zoekalgoritmes

Herman Bruno; 2023-09-15T09:38:57.730

Schuifpuzzel

De **state space** bevat configuraties van de puzzel.

De opdracht is nu om van de startpositie naar het einddoel te gaan (einddoel = opgeloste puzzel)



Deze tekening is de search tree van de state space van de schuifpuzzel